



2026 年全国大学生计算机系统能力大赛  
操作系统设计赛（全国）OS 功能挑战赛道

# SLIM-ARC：面向内存受限端侧系统的 MoE 推理运行时设计与跨设备评测

赛题：Proj59 内存受限环境的大语言模型推理优化

组员：欧阳易芃、马福泉、刘昊

指导教师：赵帅、张献伟

学校：中山大学

2026 年 8 月

# 目录

|          |                          |           |
|----------|--------------------------|-----------|
| <b>1</b> | <b>引言</b>                | <b>2</b>  |
| <b>2</b> | <b>背景、相关工作与研究动机</b>      | <b>3</b>  |
| 2.1      | 端侧推理的内存层次 . . . . .      | 3         |
| 2.2      | 相关工作 . . . . .           | 3         |
| 2.3      | 研究动机 . . . . .           | 3         |
| <b>3</b> | <b>SLIM-ARC 系统设计</b>     | <b>4</b>  |
| 3.1      | 总体架构 . . . . .           | 4         |
| 3.2      | 阶段感知页访问 . . . . .        | 4         |
| 3.3      | MoE 专家预测与反馈闭环 . . . . .  | 4         |
| 3.4      | 压力感知驻留与统一预算 . . . . .    | 5         |
| 3.5      | KV cache 与长上下文 . . . . . | 5         |
| <b>4</b> | <b>系统实现</b>              | <b>7</b>  |
| 4.1      | llama.cpp 集成 . . . . .   | 7         |
| 4.2      | 关键模块 . . . . .           | 7         |
| 4.3      | 跨设备实现 . . . . .          | 7         |
| <b>5</b> | <b>实验评估与结果分析</b>         | <b>9</b>  |
| 5.1      | 实验设置 . . . . .           | 9         |
| 5.2      | 主实验 . . . . .            | 9         |
| 5.3      | 消融与专项实验 . . . . .        | 10        |
| 5.4      | 结果总结 . . . . .           | 11        |
| <b>6</b> | <b>总结与未来工作</b>           | <b>12</b> |
| <b>A</b> | <b>主要实验数据汇总</b>          | <b>13</b> |
| <b>B</b> | <b>复现入口</b>              | <b>13</b> |

# 1 引言

大语言模型的参数规模和上下文长度持续增长，模型权重、KV cache、临时张量与操作系统页缓存共同构成了显著的运行时内存压力。MoE 模型只为每个 token 激活少量专家，降低了实际计算量；但完整权重仍需存放在本地介质，稀疏计算也不会自动转化为稀疏物理内存。对于文件大小 48.41 GB 的 Qwen3-Next-80B-A3B Q4\_K\_M，2-8 GiB 的端侧环境无法让全部权重常驻，推理性能取决于运行时如何组织映射、预取、回收、专家驻留与 KV cache。

SLIM-ARC 面向 CPU 为主的端侧设备，将模型访存视为操作系统、运行时和模型结构共同决定的问题。系统不修改权重与 Router 语义，而是在 llama.cpp 的 mmap 路径上增加阶段感知的页建议、分层预取、专家反馈闭环、压力感知预算与 KV 管理。统一调度器在 Prefill 和 Decode 之间切换策略，并依据设备的内存压力、存储介质和缓存状态选择合适配置。

本文的主要贡献如下。

- 提出阶段感知的权重访问控制方法，将 mmap、CPU repack、页建议、层级预取和设备存储特性纳入统一运行时。
- 设计 Router 反馈驱动的 MoE 专家管理闭环，以轮次标识关联预测和事实，并通过安全页范围、热度、压力与预算控制专家页面。
- 实现模型所有权的运行时、租约化访问、有界队列和统一指标，使后台任务与模型映射具有明确生命周期。
- 在 x86、RK3588、Raspberry Pi 5 和 Mac/Colima 上完成跨设备评测；48.41 GB 模型能够在 2 GiB、no-swap 条件下推理，RK3588 3 GiB 下 Decode 从 0.70 提升至 2.21 token/s。

## 2 背景、相关工作与研究动机

### 2.1 端侧推理的内存层次

端侧推理的内存需求由权重、KV cache、计算临时量和页缓存共同组成。mmap 只建立虚拟地址映射，首次访问仍需通过 major page fault 从存储读取。Linux cgroup v2 的 `memory.max`、`memory.current` 与 `memory.peak` 为实验提供了可复现的物理内存边界。`madvise` 与 `posix_madvise` 可以表达 SEQUENTIAL、NORMAL、RANDOM、WILLNEED 和 DONTNEED 等访问建议，为运行时控制 readahead 与页缓存提供了标准接口。

MoE Router 根据 token 表征选择 top- $k$  专家。稀疏专家访问形成了潜在的小工作集，但选择结果产生之前只能依赖历史、跨层或在线转移信息进行预测。专家切片还可能与相邻专家共享边界页，因此页面回收必须验证张量布局、专家编号、地址溢出和完整内部页范围。

### 2.2 相关工作

FlexInfer 研究端侧张量卸载、保留和异步预取；PowerInfer-2 通过细粒度缓存与预取支撑手机推理；KTransformers 将 MoE 热冷专家映射到 CPU/GPU 异构执行。FlashAttention、PagedAttention 与 StreamingLLM 分别从 Attention I/O、KV 分页和长序列窗口控制降低运行时开销。SLIM-ARC 吸收这些系统的预算化和异步化思想，但主要面向 CPU-only、POSIX mmap 与普通端侧存储，使机制在没有专用加速核或直接存储通路时仍可工作。

### 2.3 研究动机

本项目聚焦四个相互关联的问题：第一，Prefill 的大范围扫描与 Decode 的稀疏复用需要不同页策略；第二，专家预测必须与实际 Router 选择关联，才能管理投机 I/O；第三，普通权重、专家和 KV cache 必须共享统一资源预算；第四，后台预取线程必须服从模型映射的生命周期。对应地，SLIM-ARC 采用阶段控制器、专家反馈闭环、压力准入和模型所有权运行时，形成从访问预测到资源结算的完整链路。

### 3 SLIM-ARC 系统设计

#### 3.1 总体架构

图1展示系统的五层结构。模型层保留原始 GGUF 权重和 Router 语义；运行时层识别阶段并生成页访问动作；操作系统层执行 mmap、页建议与 cgroup 约束；存储层承载远大于物理内存的模型文件；指标层记录吞吐、缺页、I/O、预算和专家状态。统一调度器连接普通权重、专家与 KV 路径，避免各模块独立扩大工作集。

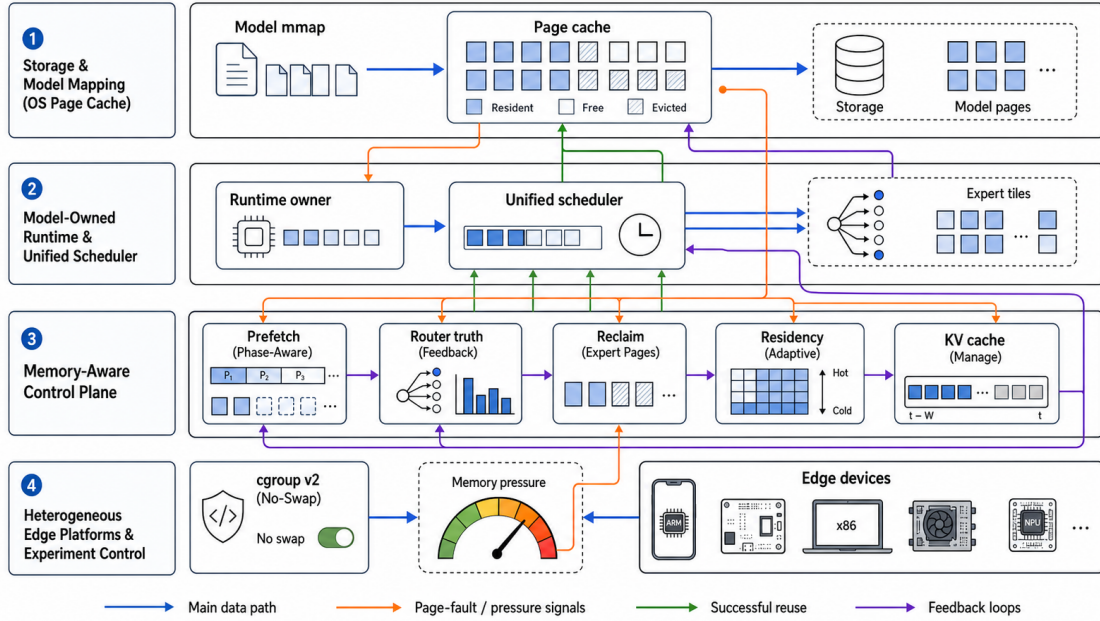


图 1: SLIM-ARC 的分层系统架构。

#### 3.2 阶段感知页访问

系统将执行划分为模型加载、Prefill、普通 Decode 和 MoE Decode。Prefill 按层扫描较大的权重集合，优先使用顺序预读并提前有限层数；Decode 重复访问少量活跃页面，预取窗口随预算收缩；MoE Decode 进一步利用 Router 产生的稀疏专家集合。策略切换只改变访问提示，不改变张量内容与计算顺序。

页建议采用 best-effort 语义。所有地址先转换为页边界范围，长度和地址运算进行溢出检查；建议调用在状态锁之外执行，避免慢系统调用阻塞 Router 与调度线程。运行时记录成功建议字节而非请求字节，使指标能够对应实际内核动作。

#### 3.3 MoE 专家预测与反馈闭环

系统支持 temporal、跨层 Router 和热度候选。每次专家预取分配不可复用的 generation token，预取成功的专家编号和字节数以 token 为键保存；实际 Router 结果到达后只结算一次。图2展示跨层预测路径：第  $N$  层 Router 的选择转化为下一层候选，在第  $N + 1$  层计算之前向内核表达 WILLNEED。

## MoE Router Hook cross-layer expert prediction

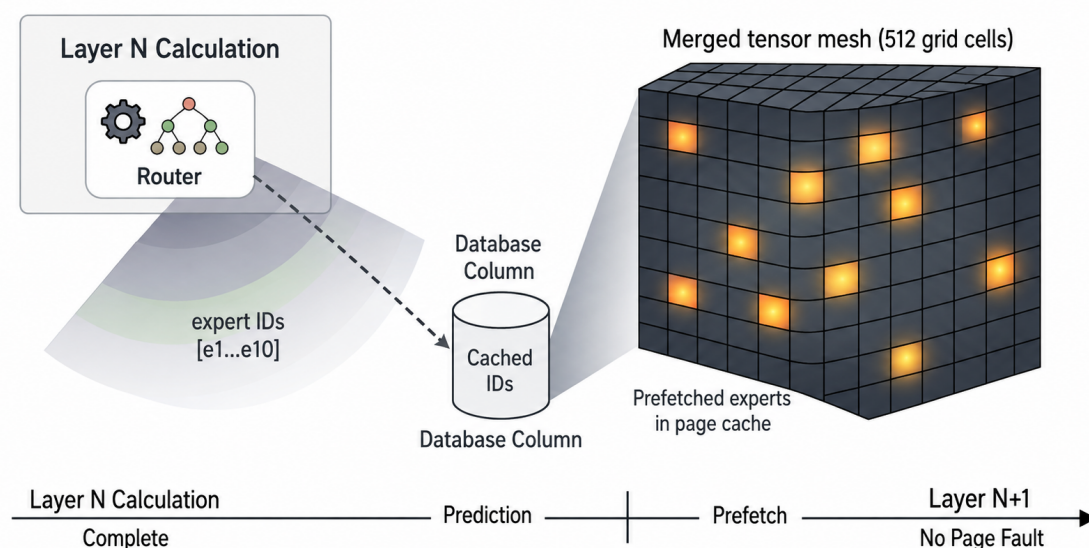


图 2: 跨层 Router 预测与专家页面预取。

事实结算后，运行时更新专家热度、命中和浪费统计。对未选择专家，只对完全位于切片内部的页面生成 DONTNEED 计划；首尾共享页保持不动。回收和驻留均受特性开关与预算约束，因此核心推理路径能够独立运行。

### 3.4 压力感知驻留与统一预算

压力控制器以 cgroup 当前使用量与上限计算状态，并使用滞回避免阈值附近频繁切换。正常压力下按稳定专家、时间相邻专家和热专家排序；较高压力下只保留确定性较强的稳定集合；临界压力下停止新增驻留。浪费比例使用整数 EWMA 平滑，专家热度定期衰减，避免早期请求永久支配缓存。

统一预算先扣除安全预留，再分配给普通权重、专家和 KV 路径。所有加法、乘法和字节累计都使用饱和或显式检查；压力信息缺失时选择保守配置。该设计使内存上限成为真正的共享约束，而不是多个模块各自解释的参数。

### 3.5 KV cache 与长上下文

KV 路径结合低比特表示、attention sink 与最近窗口。图3中，固定保留 sink token 与最近窗口，中间区域按策略释放或映射到临时文件，使 KV 占用从上下文长度线性增长转为与 sink 和窗口大小相关。权重页与 KV 预算由同一调度器协调，从而在长上下文下保留模型权重工作集的空间。

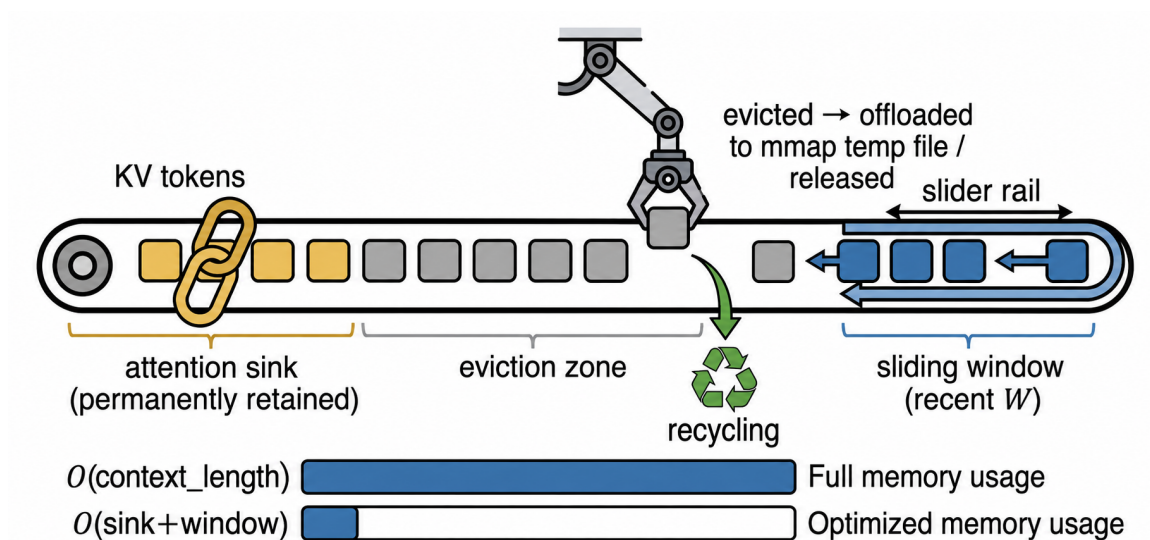


图 3: attention sink、滑动窗口与 KV 页面管理。



## 4 系统实现

### 4.1 llama.cpp 集成

项目以固定 llama.cpp 提交为基线，通过可重复应用的补丁脚本复制独立模块并插入构建目标。模型对象在完成 mapping 转移后创建运行时，运行时内部依次持有预取调度器与统一调度器；析构顺序保证 worker 停止后映射才被释放。图4给出主要模块和调用关系。

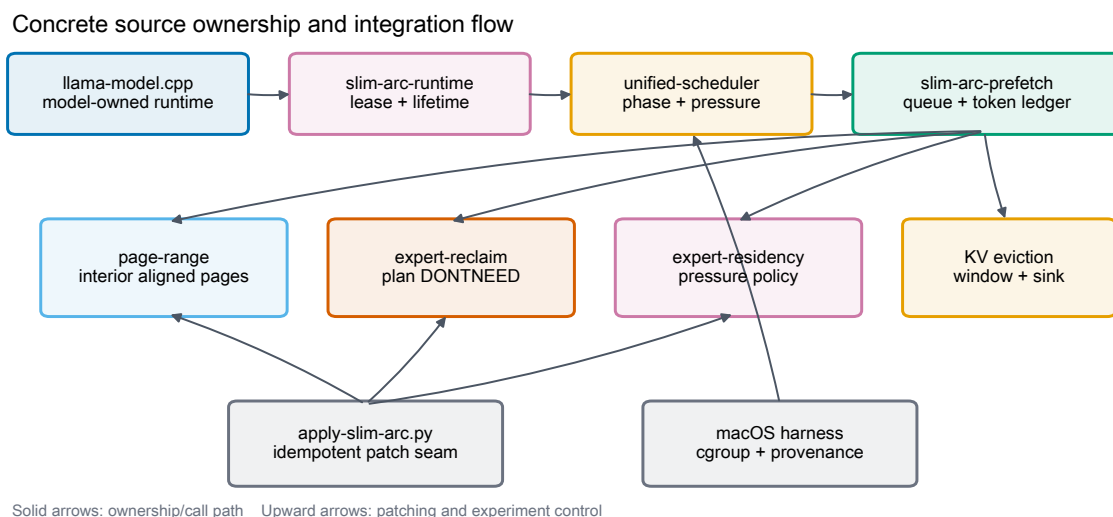


图 4: 运行时模块、模型所有权与图执行钩子的实现关系。

图执行只通过运行时租约访问调度器。租约在模型析构前登记关闭状态，随后等待在途访问退出；任何新请求都会得到空租约。该机制消除了进程全局裸指针与映射地址分离的问题。预取 worker 使用有界请求队列和单调 generation，重复层通知、图失败与无 Router 节点均有确定的终态结算。

### 4.2 关键模块

| 模块                         | 职责                             |
|----------------------------|--------------------------------|
| slim-arc-runtime           | 模型所有权、租约、mapping 与张量注册、调度器生命周期 |
| slim-arc-prefetch          | 阶段识别、普通权重与专家预取、有界队列、运行时指标      |
| slim-arc-unified-scheduler | 压力采样、共享预算、权重/专家/KV 协调          |
| slim-arc-page-range        | 页对齐、内部完整页与溢出安全计算               |
| slim-arc-expert-reclaim    | 错误专家集合、回收计划和成功字节统计             |
| slim-arc-expert-residency  | 压力状态、浪费 EWMA、热度衰减和确定性准入        |
| slim-arc-kv-eviction       | sink、滑动窗口和 KV 页面释放             |

### 4.3 跨设备实现

x86/WSL 用于建立 Qwen3-Next-80B 的受限内存基线；RK3588 使用 systemd user scope 与 cgroup v2 构造 2.5–5 GiB 边界，并针对 ARM CPU 与 SSD 调整阶段页建议；Raspberry Pi 5 使用 4 GiB 内存和 USB/FUSE 存储，重点保留 shared/hot 专家并严格限制投机 I/O；Mac 通过 Colima



ARM64 VM、Docker cgroup v2 和 `swap.max=0` 形成可重复的 2 GiB、4 vCPU 环境。所有平台保持模型哈希、缓存状态、Prompt/Generation 长度与线程数随实验记录。

## 5 实验评估与结果分析

### 5.1 实验设置

主模型为 Qwen3-Next-80B-A3B-Instruct Q4\_K\_M, 文件大小 48,410,988,384 B, SHA-256 为 d103b2733ec1012a52d01edda66b7e5c24ae50508c9f99f5297ea459ef3c061a。主要指标包括 llama-bench Prefill/Decode token/s、完整 wall time、cgroup peak、major faults、文件输入字节、专家建议字节和缓存命中。提升率只在同设备、同模型、同 workload、相同线程与缓存状态内计算。

表 2: 主要实验平台与目标

| 平台             | 资源约束                   | 存储       | 主要目标            |
|----------------|------------------------|----------|-----------------|
| x86/WSL        | 8–32 GiB cgroup        | NVMe     | 初赛基线与完整优化栈      |
| RK3588         | 2.5–5 GiB user scope   | SSD      | 3 GiB 主实验与动态页建议 |
| Raspberry Pi 5 | 4 GiB, no-swap         | USB/FUSE | 慢存储驻留与 I/O 预算   |
| Mac/Colima     | 2 GiB, 4 vCPU, no-swap | APFS/VM  | 80B 极限可运行性      |

### 5.2 主实验

**2 GiB 环境中的 80B 推理。** Mac/Colima 在 2 GiB、4 vCPU、no-swap、pp64/tg16 条件下完成 48.41 GB 模型推理。终态运行的 Prefill 为 3.908455 token/s, Decode 为 0.709095 token/s, wall time 58.06 s。该结果验证了 mmap 与受控页缓存能够让模型文件显著大于物理内存上限。

**RK3588 3 GiB 主实验。** 表3给出固定 80B、4 threads、pp128/tg64、swap 0 的结果。SLIM-ARC 在 3 GiB 下将 Prefill 从 3.85 提高到 4.40 token/s, 将 Decode 从 0.70 提高到 2.21 token/s, 对应 3.16 倍 Decode 吞吐。

表 3: RK3588 受限内存主实验

| MemMax  | 配置            | Prefill     | Decode      | 同档提升              |
|---------|---------------|-------------|-------------|-------------------|
| 3 GiB   | baseline      | 3.85        | 0.70        | reference         |
| 3 GiB   | SLIM-ARC      | <b>4.40</b> | <b>2.21</b> | pp +14%, tg +216% |
| 3 GiB   | SLIM-ARC + KV | 4.19        | 2.12        | pp +9%, tg +203%  |
| 2.5 GiB | baseline      | 3.97        | 1.91        | reference         |
| 2.5 GiB | SLIM-ARC + KV | 4.26        | 2.21        | pp +7%, tg +16%   |

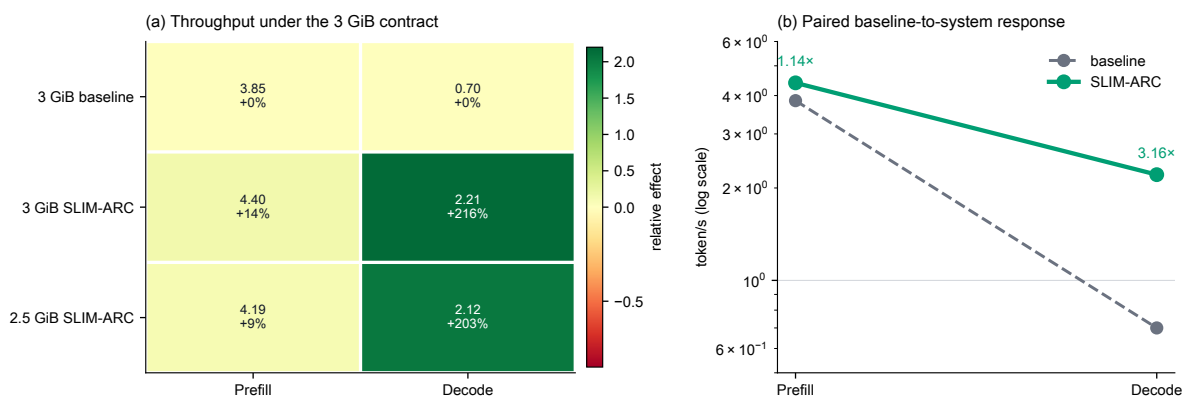


图 5: RK3588 3 GiB 主实验的归一化温度图与配对响应。

**Raspberry Pi 5 慢存储实验。** 在 4 GiB、运行期 no-swap 的 USB/FUSE 链路上，系统从扩大预取转向保留确定复用页。A28 的 shared expert 与 hot512 驻留相对 A24 将 Decode 从 0.055614 提高到 0.063486 token/s，提升 14.16%，wall time 从 547 s 降至 511 s。A29 将预算限制为 16 MiB，在吞吐近似不变的同时将投机 weight I/O 降低 98.44%。512 MiB 跨 token LRU 最终达到 0.065856 token/s 和 503 s。

表 4: Raspberry Pi 5 的慢存储策略演进

| 编号  | 策略                      | Prefill  | Decode          | Wall(s)      |
|-----|-------------------------|----------|-----------------|--------------|
| A24 | no expert prefetch      | 0.201104 | 0.055614        | 547.0        |
| A28 | shared + hot512         | 0.204023 | 0.063486        | 511.0        |
| A29 | A28 + 16 MiB budget     | 0.204283 | 0.063585        | 510.5        |
| A32 | no weight prefetch      | 0.203642 | 0.065246        | 505.0        |
| A34 | 512 MiB cross-token LRU | 0.203585 | <b>0.065856</b> | <b>503.0</b> |

### 5.3 消融与专项实验

**阶段化页建议。** RK3588 消融按阶段逐步切换访问建议，动态全 SEQUENTIAL 达到 Prefill 2.84、Decode 1.40 token/s，说明页建议应与 Prefill/Decode 阶段以及实际存储介质共同选择。

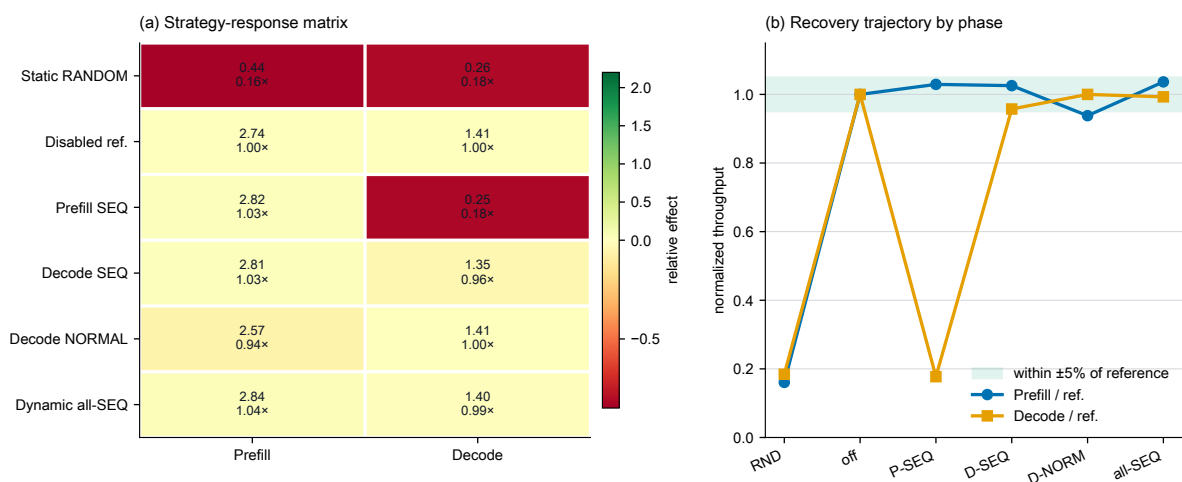


图 6: RK3588 阶段化页建议的策略响应图。

**专家预测、门控与预算。** RK3588 temporal predictor 准确率为 34.9%；confidence gating 将 expert issued 从 27,836.7 MB 降至 12,123.6 MB，hit rate 从 31.05% 提高到 55.35%，Decode 保持约 1.9 token/s。Mac A22 cross-layer Router Top2 命中率达到 88.56%，Decode 提高 9.96%，wall time 缩短 7.77%。这些结果表明，预测质量必须与投机字节和端到端时延联合分析。

**线程与 readahead。** Mac 的 8/6 Prefill/Decode 线程配置使 Decode 提高 20.23%、wall time 缩短 5.91%。持续 256 KiB readahead 配置使 Prefill 提高 13.39%、Decode 提高 2.10%、wall time 缩短 6.39%。系统将这些参数保留为设备配置，而不是写死为全平台常量。

**Attention 与长上下文。** RK3588 Qwen3-4B 上 FlashAttention auto 的 Decode 为 6.90 token/s。80B 长上下文实验在 8192–16384 context 下完成 KV eviction 路径验证；动态策略在 p512/n128 下达到 6.09/2.08 token/s。KV 优化与权重页管理分别控制上下文工作集和模型工作集，并通过统一预算协同。

## 5.4 结果总结

实验表明，SLIM-ARC 的收益来自三条主线。第一，mmap、关闭不必要的 repack 与可控页缓存使超大模型能够在小内存边界内运行。第二，阶段化页建议和设备配置使访问模式贴合实际存储链路。第三，专家预测只有与预算、热度、事实结算和安全回收结合后，才能把 MoE 稀疏激活转化为可管理的工作集。RK3588 3 GiB 与 Raspberry Pi 5 慢存储实验分别验证了强内存压力和高 I/O 代价下的策略选择。

## 6 总结与未来工作

SLIM-ARC 构建了面向内存受限端侧设备的 MoE 推理运行时。系统以阶段识别为入口，用 mmap 与动态页建议控制普通权重访问，以 Router 反馈闭环控制专家页面，以压力、热度和统一预算协调驻留、回收与 KV cache，并通过模型所有权和租约机制保证后台调度与映射生命周期一致。

跨设备实验验证了系统的核心目标：48.41 GB 模型可在 2 GiB、no-swap 环境中完成推理；RK3588 3 GiB 下 Decode 达到 2.21 token/s；Raspberry Pi 5 上确定性专家驻留与严格 I/O 预算同时改善吞吐和 wall time。结果说明，端侧推理优化的关键不是孤立扩大预取，而是依据阶段、压力和介质构造受约束的工作集。

未来工作将继续研究更长上下文下的 KV 分层、多级存储上的专家放置、在线访问模型与异构计算协同，并扩大任务质量与多轮统计规模。系统将保持权重语义不变、策略可关闭、资源边界可观测的原则。

## A 主要实验数据汇总

| 平台             | 模型与约束                                          | 配置                     | 结果                                                         |
|----------------|------------------------------------------------|------------------------|------------------------------------------------------------|
| Mac/Colima     | Qwen3-Next-80B,<br>2 GiB, 4 vCPU,<br>no-swap   | A24, pp64/tg16         | Prefill 3.908455, Decode<br>0.709095 token/s, wall 58.06 s |
| RK3588         | Qwen3-Next-80B,<br>3 GiB, 4 threads,<br>swap 0 | baseline               | Prefill 3.85, Decode<br>0.70 token/s                       |
| RK3588         | 同上                                             | SLIM-ARC               | Prefill 4.40, Decode<br>2.21 token/s                       |
| RK3588         | 同上                                             | SLIM-ARC + KV          | Prefill 4.19, Decode<br>2.12 token/s                       |
| Raspberry Pi 5 | Qwen3-Next-80B,<br>4 GiB, USB/FUSE             | A28 shared +<br>hot512 | Prefill 0.204023, Decode<br>0.063486, wall 511 s           |
| Raspberry Pi 5 | 同上                                             | A29 16 MiB budget      | 投机 weight I/O 降低 98.44%                                    |
| Raspberry Pi 5 | 同上                                             | A34 cross-token<br>LRU | Decode 0.065856, wall 503 s                                |

## B 复现入口

核心补丁位于 `patches/llama-upstream/`, `scripts/apply-slim-arc.py` 将模块应用到固定 llama.cpp 源码树。C++ 单元测试由 `tests/run-cpp-unit.sh` 执行; Mac 受限内存入口位于 `scripts/macos/`。实验复现应固定模型 SHA-256、llama.cpp 提交、内存与 swap 上限、线程数、Prompt/Generation 长度和缓存状态。

## 参考文献

- [1] H. Du et al., “FlexInfer: Breaking Memory Constraint via Flexible and Efficient Offloading for On-Device LLM Inference,” EuroMLSys, 2025.
- [2] Z. Xue et al., “PowerInfer-2: Fast Large Language Model Inference on a Smartphone,” 2024.
- [3] KTransformers, <https://github.com/kvcache-ai/ktransformers>.
- [4] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022.
- [5] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP, 2023.
- [6] G. Xiao et al., “Efficient Streaming Language Models with Attention Sinks,” ICLR, 2024.
- [7] llama.cpp, <https://github.com/ggml-org/llama.cpp>.

- [8] Linux kernel documentation, “Control Group v2,” <https://docs.kernel.org/admin-guide/cgroup-v2.html>.